

ABRA — roadmap

2026-07-26

The bar for the completed list is one thing: **would I defend this sentence for twenty hours against someone trying to break it.** Anything that fails that test is in section 2 or section 3, however much work went into it.

A result is only here if it is (a) measured rather than argued, (b) measured on data that passes `engine/quality.js` , and (c) still true when re-run today. Several things that were on this list a week ago have been removed for failing (c).

1. DONE — and defensible

1.1 The data is trustworthy, and we can prove which parts are not

87% of the ladder store is unusable and we know exactly why. 15,973 stored games, ~2,057 clean. Bots, forfeits, partial brings and stubs, each counted separately in a funnel that anyone can re-run. 27 assertions in `tests/test-quality.js` .

Behavioural bot detection works on accounts that do not announce themselves. Team invariance: an account playing 50+ games without ever changing a slot is a script. It caught six accounts that appeared in **52.2%** of the games a name-only filter called clean. Carchdraw84172 played 459 games with one team, 367 of them in a single day.

Defensible because: the rule is stated, the evidence is per-account, and the false-positive check is published — no account with ≥ 50 games in the store has more than one distinct team, so the rule separates cleanly.

Known limit, stated: a bot playing under 50 games or varying its team is invisible. The right phrase is always "no bot detected", never "human".

1.2 The damage engine is correct

31 of 31 scenarios within 2% of `@smogon/calculator` , through the official Showdown engine pinned at commit `20ad99f` . Golden-master, re-run before every engine change.

Defensible because: it is validated against an independent implementation, not against itself, and the harness caught two of its own wiring bugs on first run.

1.3 MAGNEMITE reads the board, and the improvement is measured out of sample

600 seed-matched battles per policy — same teams, same seeds, same engine — against 1,905 clean ladder games:

	before	after	real humans
super effective	9.71%	14.91%	21.37%
moves that outright failed	9.68%	6.34%	2.47%
moves that hit an immune target	4.30%	2.92%	1.91%

Defensible because: the fit never saw the realism report — it was held back as the out-of-sample check — and the comparison is paired on seeds so team difficulty cancels.

What this does NOT claim: that MAG is good. See 2.1.

1.4 Imitation is a ceiling, and we have the number

A challenger optimised for **winning** rather than for resembling humans, drawn from MAG's own 17 features, beats MAG **63.2%** — and beats MAG's predecessor **68.2%** where MAG itself manages **60.2%**.

Defensible because: the 68.2% is an independent evaluation the search never optimised against, on an opponent it was not tuned for. It is transitively better, not a counter.

And it is interpretable: the winning weights barely care about type effectiveness and care enormously about not wasting a turn and finishing what is already hurt.

1.5 The open-sheet corpus is safe to learn from, and we checked rather than assumed

Teams differ enormously between open- and closed-sheet play — **551.9 points** of total absolute species difference, Garchomp on 81.6% of open-sheet teams against 47.7% of closed. Behaviour given a board differs by **at most 1.49 points** on every measure.

Defensible because: both sides were measured with the same code, and the policy is conditional on the board — it never learns what to bring, and MEW draws teams from the ladder store regardless.

`engine/fit_policy.js` re-checks it on every refit by re-estimating on a reweighted sample.

1.6 Collection is complete and automatic

The Bo3 **open-team-sheet** ladder is now pulled hourly into its own store. That format carries `Force Open Team Sheets`, so every game publishes all six sets — the only continuously-collected corpus where the choice set of a decision is known rather than guessed.

Defensible because: the format's ruleset was read from the simulator, the pull was tested live, and the store is separate — Bo3 is a different information regime and pooling it would corrupt every behavioural statistic.

1.7 Facts the model can see are derived, never typed

Ability blocking (Levitate/Ground, Flash Fire/Fire, Good as Gold/status, Armor Tail/priority) is **read out of 14,933 recorded battles**, not written down — including inferring *which rule* each ability follows by testing candidate hypotheses and preferring the narrower on ties.

Defensible because: it was verified against clean-only data and every rule is identical; and because probing Showdown's handlers directly was tried first and **failed silently**, which is why the measured route was taken.

1.8 The project can audit itself

- `engine/provenance.js` — every artifact checked for staleness against the filter, its inputs, and the clean-game count. Found **28 unsafe to quote**.
- `engine/conformance.js` — S1–S13 as executable checks over 132 files. **S12 is now clean**.
- `engine/selftest.js` — 24 assertions on failures that are silent by nature.
- `build/build_pdfs.js` — every document has a current PDF, list derived.

Defensible because: each found real defects on its first run, including in the tool that found them.

2. NOT DONE — and specifically why

2.1 MAG is not good

It is **exploitable** (1.4), it still repeats Protect at roughly double the human rate, it wastes 6.34% of moves against a human 2.47%, and its one head-to-head win was against a baseline with a flaw it was purpose-built to punish. **It is a starting position, not a player.**

2.2 The self-play ladder has produced no evidence

One confirmed promotion. The round robin that would establish whether the ladder climbs or goes in circles has never had enough games to resolve anything. Reported as a null.

2.3 Twenty-eight artifacts are unsafe to quote

Listed by `node engine/provenance.js` . Until regenerated, nothing downstream of them means anything. **This is the blocker on everything in section 3.**

2.4 Models that are retracted, null, or without usable input

Recorded so they are not quoted again:

model	state
PORY	its comparison artifact was trained without the filter; standing unknown , not weak
SLOWKING	mathematics sound, but the clean matrix has o decisive matchups — no usable input
GURU	o decisive matchups on clean data; descriptive only
JOLTEON	ties a coin; recommended for retirement
WAR	withdrawn — the signal was four bots playing one team 1,446 times
CHOMP-EV	honest null; brings do not separate from a coin
the 199,524-game self-play corpus	unusable — generated before megas worked, with duplicate Protects on one set
"the meta is rock-paper-scissors"	withdrawn; computed pre-filter, deleted

2.5 Nothing here handles hidden information

Self-play has none — we run both sides. Nothing in the current work teaches the bot to play against an unknown team, which is the defining difficulty of the closed-sheet ladder.

2.6 Closed since this list was written (2026-07-29 → 30)

Items 3, 4 and 5 below have shipped. Recorded rather than deleted, because a roadmap that quietly drops its own entries cannot be audited.

- **"Give MAG a damage calculation, and a KO check both ways"** — DONE. `board.js` calls the real damage engine throughout; `koTarget` , `killIsRoll` , `diesBeforeMoving` and the switch-survival features all read it.
- **"Score both Pokémon together"** — BUILT AND MEASURED, and it **lost**: 42.0% [39.9, 44.3] over 1,934 paired games, 28.4% of decisive pairs, against its own zeroed control. The imitation fit is refuted; the idea is not. Now item 1 below.
- **"Switching"** — no longer a coin flip. Voluntary switches score through `switchFeatures` , the post-KO replacement is scored rather than rolled, and `entryHits = forced ? 0 : 1` separates the two cases.

A process failure that caused real damage, recorded so it is not repeated. On 2026-07-30 the model family was repeatedly mischaracterised in conversation — DODUO described as unbuilt when it had been built, wired, controlled and measured; MEDICHAM audited in its graveyard version; top-K pruning "proposed" when `fit_joint.js` already had it. Cause: two full days and ~40 commits of work existed in NO document. `MODELS.md` was last written 2026-07-28; every other living doc 2026-07-26. The only reason DODUO's verdict was findable at all is that its experiment happened on the one day `MODELS.md` was touched. **CLAUDE.md requires the docs to move in the same pass as the code. They did not.**

2.7 Closed 2026-07-31

- **"Choice lock in the candidate set"** (was item 2) — DONE. The logit denominator no longer contains moves the human could not click. After the refit, six of eight switch features clear zero, and **switches now win the argmax** where `--switching` previously changed nothing.
 - **The tag artifact reaches the feature vector** — `speedSwing` +0.983, `screenValue` +1.128, `healValue` +2.220, all clearing zero by wide margins. 53 → 56 features. This is the first feature addition since 3.28.0's four nulls to measure as real, and the difference is that these are CONDITIONS on mechanics with no prior representation rather than restatements.
 - **ALAKAZAM job 2 (guess what they will do)** — built, off by default, needs a refit. The max in `incomingThreat` is now an expectation over P(their action).
 - **A gate against wasted runs** — `engine/preflight.js`, 24 games, refuses a training run whose feature blocks receive no gradient. Two 1.5-hour runs were lost to exactly that before it existed.
-

3. NEXT — in order, with the reason each is next

1. DODUO trained for WINNING, not for resemblance. *The most precise open question in the project.* DODUO is fully built: `fit_joint.js` fits the pair block, `magnemite.js` plays it (`--joint`), `--joint-zero` is a true control running the whole pair path with only the coordination weights zeroed, top-K is capped, and fallbacks are counted in `stats.jointFallback`. It lost at 42.0%, and the fit itself says why — it prices `spreadFreeBesideAlly` at **-5.054**, which means *humans rarely click this, not this is bad*. A bot told to avoid a free spread move beside its own ally by -5 will decline its best plays.

2026-07-30 established the pattern that makes this item one: **four knowledge additions produced four nulls; two changes to the OBJECTIVE produced two large wins** (greedy +12 points / 79.7% of decisive pairs; self-play 55.9%). The objective is the binding constraint, and DODUO has only ever been fitted to the losing one.

RECONCILED 2026-07-31. *That 55.9% was measured on the 53-feature vector with switching OFF. Repeating the experiment on the 56-feature vector with switching ON gives 48.1% [46.5, 49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration being harmful (consistent with the older 10-point switching loss), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.*

The gap is small and exact. `train_policy.js` has no joint support, and `magnemite.js` 's learning gradient is sized to `this.w` (53 singles) while the joint vector is `this.wj` (53 + 21 = 74). The pair softmax is the same conditional logit and `accumulateLogitGrad(g, vecs, probs, j, nw)` is already generic — this is wiring, not a new model.

A trap already paid for once: `fit_joint` fits its single block and its pair block TOGETHER, and 23 of 48 features carry opposite signs between that fit and the shipped one. Mixing the two vectors lost 31.2% on decisive pairs and would have been reported as "coordination does not help."

2. Choice lock in the candidate set. Exact, free, and a FITTING BUG rather than only a prune. Live play is safe — Showdown's request marks the other moves `disabled` — but `fit_policy.js` hands `candidates()` all four sheet moves with no legality filter, so a choice-locked human appears to have had 9 options when they had 4, and the conditional logit's denominator contains five actions that were never available. **6.52% of items in this format.** The dex flags `choiceband`, `choicespecs` and `choicescarf` with `isChoice`, so nothing

is typed in; the board already tracks `item` and `lastMove`, and a fresh switch-in has `lastMove: ''`, so "not locked on the turn it arrives" falls out with no turn counter. **Do this BEFORE re-measuring DODUO**, or the denominator is wrong in both arms.

3. Re-run WOBBUFFET against the current vector. (*Will, 2026-07-30: put this back on the list.*) Its result is still the most important number in the repo and it is three feature-generations stale: a counter found in forty minutes beat MAG **63.2%** [56.6, 69.3] on the 17-feature vector, mirror control 47.5%. That challenger was not rock-paper-scissors — it was a better player drawn from the same features and optimised for wins rather than resemblance, which is item 1's lesson in another form. `engine/exploit.js --target <weights.json>` can now be pointed at DODUO, so the EXPLOITABILITY argument for coordination — the one the 42.0% result explicitly does not settle — can finally be tested instead of argued.

4. MACHAMP, re-run on the current vector. (*Will, 2026-07-30: keep it.*) Half-run and stale: the 2026-07-26 run completed 2 of 6 generations on a **17-feature** vector and recorded no verdict; the vector is now 53. Its METHOD is alive — champion/challenger promotion behind a Wilson interval is the same win-objective idea `train_policy.js` now implements by policy gradient. Keep the guard that made it honest: every promoted champion plays EVERY previous generation, because this metagame is cyclic and "gen 5 beats gen 4" is not progress.

5. Branch scoring with a mixed strategy — the research is done, so this is no longer vague. Measured on a real mid-game board: **$9 \times 8 = 72$ joint actions per side, $72 \times 72 = 5,184$ matrix cells.** The 10^{12} figure from VGC-Bench is the game TREE, not one turn.

- Only **28 of 72** pairs have a non-zero joint vector. The other **44 are free** — their score is exactly the sum of two singles already computed.
- With two Pokémon the coordination graph is a SINGLE EDGE, so Variable Elimination reduces to enumerating the 72 pairs and Max-Plus is unnecessary. The MARL factorisation literature (QMIX, QPLEX, Weighted QMIX) exists to avoid enumeration at many agents, which is not our problem.
- What that literature DOES tell us: QMIX's monotonicity constraint cannot represent non-monotonic coordination — exactly "Protect while my partner removes the threat." Independent per-slot scoring cannot express it either. Not "scores it badly": cannot express it.
- Pruning order matters. Structural prunes (choice lock, spread moves with no target, immunities) are exact and happen BEFORE scoring. Score-based top-K is circular for the 72 and only pays for the 5,184.
- Pokémon is SIMULTANEOUS-move, so the answer at a node is a mixed strategy, not an argmax. Minimax is the wrong algorithm; this is poker-shaped. SLOWKING's regret matching already does this mathematics at team preview and nobody has pointed it at a turn.

6. A leaf evaluator, so depth > 1 is affordable. `position_features.js` (16 features) plus PORY retrained on clean data. Without it every node rolls to the end; with it, depth 2 is 625 cheap evaluations.

7. Retire the second scorer in `app/index.html`. It assigns **21 of 53 features**; 32 are never written and silently read 0, and 7 of the 21 disagree with the engine outright. This is the "facts are global" rule broken inside the UI. `app/scoreboard.html` already does it correctly by rendering engine-computed values. The real fix is to ship the actual engine to the browser — `medicham2-browser.js` already runs in both, and `board.js` has no top-level requires; the only blocker is one `process.env` read at module scope.

8. Bring the five living docs current, and keep them current. Whitepaper, deck, technical docs, SUMMARY and MODELS are at 3.21.0 against a CHANGELOG at 3.28.0. Not a version bump — see the process failure recorded in 2.6.

9. Measure exploitability again, against the mixed-strategy version. The test that says whether the project has moved from *plays okay* onto the branch that can approach *solved*.

9. External validation. Our bots grading each other cannot establish competence. The two real scoreboards are the Bo3 open-sheet ladder — where the hidden-information problem does not exist and we already collect the games — and VGC-Bench's published agent results on that same format.

The honest summary

What is genuinely strong: the data discipline, the validated engine, the measurement culture, and the tooling that now enforces all three. The project's most valuable output to date is not a model — it is the ability to tell which of its own results are real, which was demonstrated by dissolving four of them in a single day.

What is not: every in-battle model. MAG is a competent starting position and a measurably exploitable one. No model in this project has yet been shown to play VGC well against anything other than a worse version of itself.

The gap between those two paragraphs is the whole roadmap.